

ODE 용 최종 Mu Input 전달 요약

목적

이 폴더는 이미지 팩터 실험 결과를 바탕으로 ODE 포트폴리오 최적화 팀에 넘길 ``mu(t)``, ``Sigma(t)``, realized return 입력값을 한 곳에 모은 전달 패키지다.

Optimization과 ODE solver 실행은 아직 포함하지 않았다. 여기서는 ``date-asset``별 expected-return ranking signal과 ODE 검증에 필요한 covariance/return 자료를 제공한다.

폴더 위치

레포 루트:

```
```text
/home/jonghan/findalpha/delta/sprint2
```
```

최종 전달 패키지:

```
```text
ode_inputs_cnn/ode_team_handoff_20260519/
```
```

ODE 팀이 먼저 보면 되는 파일:

```
```text
ode_inputs_cnn/ode_team_handoff_20260519/02_mu_inputs/selected_mu_input.csv
```
```

최종 추천 Input

| 항목 | 내용 |
|--------------------|--|
| 추천 input | <code>`mu_image_factor_rank`</code> |
| 원천 모델 | <code>`ensemble_4family+image_factor_pc1`</code> |
| 이미지 variant | <code>`ohlcv_ma_volume`</code> |
| 의미 | OHLC candle + moving average + volume 이미지에서 추출한 image factor를 기존 4-family ensemble에 추가 |
| signal type | date별 cross-sectional centered rank |
| lookback / horizon | 60 / 20 |
| OOS rows | 20,160 |
| OOS dates | 2,880 |
| assets | 7 |

원자료 Baseline 대비 어느 정도인가

여기서 말하는 원자료 baseline은 CNN/LSTM/image factor가 전혀 없는 ``logistic_cumulative_scale``이다. 즉, ETF OHLC 원자료를 누적수익률 스타일로 scaling한 뒤 단순 logistic/linear baseline을 적용한 것이다.

| signal / model | 의미 | rank corr | Sharpe | 누적수익 | hit rate |
|--|--------------------------------------|-----------|--------|---------|----------|
| <code>`logistic_cumulative_scale`</code> | 이미지 없음, CNN/LSTM 없음, 원자료 기반 baseline | -0.0072 | 0.0764 | -0.0512 | 0.5556 |
| Equal-weight ETF buy-and-hold | 모델 없음, 7개 ETF 동일비중 보유 | n/a | 0.2086 | 0.2186 | 0.5257 |

```
| `logistic_image_scale` | 단순 logistic 이지만 image-style scaling 사용 | 0.0392 |
0.3850 | 0.6569 | 0.5903 |
| `ensemble_4family` | image factor 추가 전 rank-corr baseline | 0.0674 | 0.5430 |
1.2395 | 0.6042 |
| `ensemble_4family+image_factor_pc1` | 최종 추천 image-factor ranking input |
0.0798 | 0.3155 | 0.5517 | 0.5694 |
| `strict-window ensemble_4family+image_factor_pc1` | MA를 각 60일 window
내부에서만 계산한 보수적 image-factor 후보 | 0.0771 | 0.6053 | 1.4324 | 0.5833 |
```

해석:

- 원자료 baseline 인 `logistic\_cumulative\_scale`은 rank corr가 거의 0보다 낮아 예측력이 약하다.
- 같은 단순 logistic 이어도 image-style scaling을 쓰면 rank corr가 0.0392로 올라간다.
- 최종 추천 input은 기존 `ensemble\_4family`보다 rank corr를 0.0674에서 0.0798로 올린다.
- strict-window MA 후보도 rank corr를 0.0771까지 올리며, 기존 full-series MA보다 Sharpe trade-off가 작다.
- 다만 simple top-k Sharpe는 낮아지므로, 이 신호는 직접 매매전략이 아니라 ODE의 ` $\mu(t)$ ` ranking input으로 검증해야 한다.

왜 이걸 추천하는가

기존 `ensemble\_4family` 대비 rank correlation이 개선됐다.

```
candidate	rank corr	Sharpe	rank corr lift	bootstrap CI
`ensemble_4family`	0.0674	0.5430	0.0000	baseline
`ensemble_4family+image_factor_pc1`	0.0798	0.3155	0.0124	[0.00001,
0.02505]				
`strict-window ensemble_4family+image_factor_pc1`	0.0771	0.6053	0.0097	
[-0.00290, 0.02195] |
```

즉, 단순 top-k Sharpe는 낮아졌지만 cross-sectional ranking 성능은 좋아졌다. ODE의 ` $\mu(t)$ `가 자산 간 상대 기대수익 ranking 역할을 한다면 이 input을 실험해볼 가치가 있다.

strict-window MA 후보는 moving average 선이 60일 image window 바깥의 과거정보를 압축한다는 우려를 줄이기 위한 보수적 robustness input이다. CI가 0을 포함하므로 엄격한 통계적 개선으로 표현하면 안 되지만, rank corr 개선이 유지되고 Sharpe가 더 안정적이어서 ODE 비교 후보로 함께 넘기는 것이 적절하다.

같이 넘기는 비교 후보

```
input_name	source_model_name	역할
`mu_image_factor_rank`	`ensemble_4family+image_factor_pc1`	최종 추천 ranking
개선형		
`mu_image_factor_strict_rank`	`ensemble_4family+image_factor_pc1`	strict-
window MA 기반 보수적 robustness 후보		
`mu_image_factor_balanced`	`ensemble_best+image_factor_pc1`	image factor
추가, Sharpe trade-off가 작은 비교 후보		
`mu_rank_baseline`	`ensemble_4family`	image factor 추가 전 rank-corr
baseline		
`mu_sharpe_baseline`	`ensemble_best`	Sharpe 안정형 baseline
```

사용할 파일

```

파일	용도
`02_mu_inputs/selected_mu_input.csv`	추천 input 하나만 들어있는 파일. ODE 팀은
여기서 시작하면 된다.	
`02_mu_inputs/final_mu_inputs_long.csv`	5개 후보 전체, long format
`02_mu_inputs/final_mu_inputs_wide.csv`	5개 후보 전체, wide format
`02_mu_inputs/final_mu_inputs_calibrated.csv`	expanding-window 회귀로 return-
scale로 보정한 `mu(t)` 후보	
`03_sigma_returns/sigma_wide.csv`	`mu(t)` 날짜와 정렬된 일별 `Sigma(t)` wide
format	
`03_sigma_returns/sigma_long.csv`	`date-asset_i-asset_j` 단위의 full
covariance matrix long format	
`03_sigma_returns/returns_for_ode.csv`	`mu(t)`, `Sigma(t)`와 정렬된 실제 일별
수익률	
`02_mu_inputs/input_performance_summary.csv`	후보별 성능 요약
`02_mu_inputs/main_vs_strict_mu_comparison.csv`	기존 main image factor 와
strict-window MA 후보 직접 비교	
`02_mu_inputs/mu_calibration_summary.csv`	calibrated `mu`의 coverage, RMSE,
rank corr 요약	
`01_start_here/README.md`	영어 설명, 컬럼 설명, 사용법
`01_start_here/manifest.json`	machine-readable metadata
`03_sigma_returns/sigma_manifest.json`	Sigma 관련 metadata

```

Sigma(t)도 있는가?

있다. 최종 전달 패키지 안에 `Sigma(t)`도 같이 정리해두었다.

```

파일	크기/형태	설명
`03_sigma_returns/sigma_wide.csv`	2,880 rows x 29 columns	날짜별 7개
variance와 21개 covariance		
`03_sigma_returns/sigma_long.csv`	141,120 rows x 5 columns	날짜별 7 x 7
covariance matrix 전체		
`03_sigma_returns/returns_for_ode.csv`	2,880 rows x 8 columns	실제 일별
수익률, backtest/realized PnL 용 |

```

Sigma 설정:

- 원천 파일: `ode\_inputs\_cnn/ensemble\_best/ode\_bundle.csv`
- 스케일: daily log-return covariance
- rolling window: 60 trading days
- 날짜 범위: 2014-09-24 ~ 2026-03-09
- 자산 수: 7
- 결측치: 없음

주의할 점은 Sigma는 image factor 모델에서 새로 예측한 값이 아니라, 과거 수익률 기반 rolling covariance다. 즉 ODE 팀은 `02\_mu\_inputs/selected\_mu\_input.csv`에서 `mu(t)`를 받고, `03\_sigma\_returns/sigma\_wide.csv` 또는 `03\_sigma\_returns/sigma\_long.csv`에서 `Sigma(t)`를 받으면 된다.

패키지 내부 근거 파일 위치

```

목적	파일
전체 패키지 안내	`README_ODE_TEAM.md`
ODE 용 시작 README	`01_start_here/README.md`
image factor Phase 1-2 요약	
`05_source_notes/image_factor_phase12_final_report.md`	
Strict-window MA robustness 결과	
`05_source_notes/strict_window_ma_result_note.md`	
30개 ETF pretraining negative result	
`05_source_notes/extra_pretraining_result_note.md`	
Multiple-testing 보정 결과	`04_validation_reports/multiple_testing_report.md`
Multiple-testing 전체 CSV	
`04_validation_reports/image_factor_significance_multiple_testing.csv`	
Subperiod stability 결과	
`04_validation_reports/mu_subperiod_stability_report.md`	
Subperiod stability CSV	`04_validation_reports/mu_subperiod_stability.csv`
Return-scale calibration 결과	
`04_validation_reports/mu_calibration_report.md`	
Strict-window MA ODE 후보 비교	`02_mu_inputs/main_vs_strict_mu_comparison.csv`

```

중요한 컬럼

```

컬럼	설명
`date`	signal date
`asset`	ETF asset
`input_name`	ODE input 후보 이름
`source_model_name`	원천 signal/model 이름
`mu_signal`	ODE 에 넣을 기본 signal. date 별 centered rank
`mu_raw_score`	원래 ensemble/image-factor score
`mu_rank`	date 별 percentile rank
`mu_centered_rank`	`mu_rank`를 date 별 평균 0으로 만든 값
`mu_zscore`	date 별 z-score
`mu_calibrated_horizon`	expanding-window 회귀로 보정한 20 일 기대수익률.
`final_mu_inputs_calibrated.csv`에 있음	
`mu_calibrated_daily`	`mu_calibrated_horizon / 20`으로 만든 일별 기대수익률
`future_return`	평가용 realized 20-day return. ODE 입력으로 쓰면 안 됨
`chart_variant`	image factor 의 chart 구성
`strict_window_ma`	`True`이면 MA 선을 각 60 일 image window 내부 가격만으로 계산한
보수적 후보 |

```

ODE 팀 사용법

추천 첫 실험:

1. `02\_mu\_inputs/selected\_mu\_input.csv`를 읽는다.
2. `date × asset` wide matrix로 pivot 한다.
3. `mu\_signal`을 `mu(t)` 후보로 사용한다.
4. `03\_sigma\_returns/sigma\_wide.csv` 또는 `03\_sigma\_returns/sigma\_long.csv`에서 `Sigma(t)`를 읽는다.
5. 비교 대상으로 `mu\_sharpe\_baseline`, `mu\_rank\_baseline`,

`mu\_image\_factor\_balanced`, `mu\_image\_factor\_strict\_rank`도 돌린다.

만약 ODE 코드가 return 단위의 `mu(t)`를 요구하면 `final\_mu\_inputs\_calibrated.csv`의 `mu\_calibrated\_daily` 또는 `mu\_calibrated\_horizon`을 사용한다. 다만 calibration은 단위 문제를 줄이는 보조 layer이며, 추정오차가 추가되므로 기본 실험은 rank 기반 `mu\_signal`부터 시작하는 것을 권장한다.

간단한 pandas 예시:

```
```python
import pandas as pd

mu = pd.read_csv("ode_inputs_cnn/ode_team_handoff_20260519/02_mu_inputs/
selected_mu_input.csv", parse_dates=["date"])
mu_wide = mu.pivot(index="date", columns="asset", values="mu_signal")

sigma = pd.read_csv("ode_inputs_cnn/ode_team_handoff_20260519/03_sigma_returns/
sigma_wide.csv", parse_dates=["date"])
returns =
pd.read_csv("ode_inputs_cnn/ode_team_handoff_20260519/03_sigma_returns/
returns_for_ode.csv", parse_dates=["date"])
```
```

결론

ODE 팀에는 우선 `02\_mu\_inputs/selected\_mu\_input.csv`의 `mu\_signal`을 `mu(t)` 후보로 넘기면 된다. `sigma(t)`는 `03\_sigma\_returns/sigma\_wide.csv` 또는 `03\_sigma\_returns/sigma\_long.csv`를 쓰면 된다. 메인 input은 원자료 baseline보다 훨씬 강하고, 기존 `ensemble\_4family`보다 rank corr가 높다. 추가로 `mu\_image\_factor\_strict\_rank`는 MA 계산을 더 보수적으로 제한한 robustness 후보이므로, ODE optimizer 안에서 main input과 함께 비교하는 것이 좋다.